

Module 5 & 6: Spring Security, JWT, OAuth2, Authorization, and Exception Handling

Questions

Q1. What is Spring Security, and why is it important in a Spring Boot application?

Answer: Spring Security is a powerful, customizable authentication and access-control framework for Java applications. In a Spring Boot project, adding `spring-boot-starter-security` auto-configures basic security, including a default form login and password-based authentication. It protects REST endpoints, method invocations, and even view-layer resources. It is important because security cannot be bolted on later; it must be part of the design, from request filtering to method-level authorization and secret management.

Q2. How does Spring Security work internally at a high level?

Answer: Spring Security works through a chain of servlet filters known as the `SecurityFilterChain`. Every incoming request passes through this chain, where filters handle tasks such as extracting credentials, checking sessions, validating CSRF tokens, and enforcing access rules. The result is stored in the `SecurityContext`, which is held by `SecurityContextHolder`. Later, authorization decisions consult this context. In modern Spring Boot 3 applications, you configure security by exposing a `SecurityFilterChain` bean instead of extending `WebSecurityConfigurerAdapter`, which is now removed.

Q3. What are the core components of Spring Security?

Answer: The core components are `SecurityContext`, `SecurityContextHolder`, `Authentication`, `AuthenticationManager`, `ProviderManager`, and `AuthenticationProvider`. `SecurityContext` stores the currently authenticated user. `SecurityContextHolder` carries it across the application, typically bound to the current thread. `Authentication` represents the credential token. `AuthenticationManager` delegates validation to `ProviderManager`, which tries each registered `AuthenticationProvider` until one succeeds. `UserDetailsService` loads user data, and `PasswordEncoder` hashes passwords. Understanding how these pieces fit together is essential for debugging 403 and 401 errors.

Q4. What is the difference between `AuthenticationManager` and `ProviderManager`?

Answer: `AuthenticationManager` is an interface that defines a single `authenticate` method. `ProviderManager` is its most common implementation. It maintains a list of

`AuthenticationProvider` implementations and asks each one whether it can authenticate the given token. For example, one provider might handle username-and-password tokens, while another handles JWT tokens. If no provider supports the token or all fail, the authentication fails. This design lets you plug in different authentication mechanisms without changing the rest of the application.

Q5. What is `SecurityContextHolder`, and how does it relate to `SecurityContext`?

Answer: `SecurityContextHolder` is a strategy-based container that holds the `SecurityContext` for the current execution context. By default it uses thread-local storage, so each request thread has its own context. The `SecurityContext` contains the `Authentication` object, which includes the principal, credentials, and granted authorities. This is how controllers and services later call `SecurityContextHolder.getContext().getAuthentication()` to know who is logged in. In asynchronous or reactive scenarios, the strategy must be updated to propagate context across threads.

Q6. What is a `SecurityFilterChain`, and how do you configure it in Spring Boot 3?

Answer: A `SecurityFilterChain` is a bean that defines the order and behavior of security filters for a set of requests. In Spring Boot 3, you expose it as a bean instead of overriding `WebSecurityConfigurerAdapter`. You use `HttpSecurity` to define which endpoints are public, which require authentication, which require specific roles, and how login, logout, CSRF, and session management behave. A typical configuration disables CSRF for stateless APIs, requires authentication for protected paths, and sets the session creation policy to stateless when using JWT.

Q7. What is CSRF, and how does Spring Security protect against it?

Answer: Cross-Site Request Forgery is an attack where a malicious site tricks a logged-in user's browser into making an unwanted request to a trusted site. Spring Security protects browser-based sessions by issuing a random CSRF token per session and requiring it for state-changing requests such as POST, PUT, DELETE, and PATCH. The token is typically rendered in forms or returned as a cookie/header for JavaScript clients. For pure REST APIs consumed by mobile apps or other backends, CSRF is usually disabled because the client does not maintain a session cookie and the token is not practical to manage.

Q8. When is it safe to disable CSRF, and what is the risk of disabling it globally?

Answer: It is safe to disable CSRF for stateless APIs where authentication is carried by a header

token such as JWT, because the browser does not automatically send custom headers. It is dangerous to disable CSRF globally for browser-based applications that rely on session cookies, because an attacker can forge requests using the victim's stored credentials. If you must expose both browser forms and APIs from the same application, configure CSRF per request matcher rather than turning it off everywhere.

Q9. What is XSS, and how do you prevent it in a Spring Boot application?

Answer: Cross-Site Scripting is an attack where an attacker injects malicious scripts into web pages viewed by other users. In a Spring Boot REST API, the main defense is never trusting user input and always escaping output. Use templating engines that auto-escape content. For JSON APIs, ensure the client escapes data before rendering it in HTML. Set a strong Content Security Policy header, use `HttpOnly` and `SameSite` cookies, and validate input on the server. Storing raw HTML from users and rendering it later is a common way XSS vulnerabilities appear.

Q10. What is SQL injection, and how does Spring Data JPA help prevent it?

Answer: SQL injection is an attack where an attacker inserts malicious SQL into input fields to manipulate database queries. Spring Data JPA helps prevent it because JPQL and named parameters use prepared statements, separating code from data. A query written as `SELECT u FROM User u WHERE u.email = :email` is safe, while concatenating user input into a native query is not. Even with JPA, native queries and dynamic SQL must use placeholders or the Criteria API. Input validation and least-privilege database accounts provide additional defense.

Q11. What is the difference between authentication and authorization?

Answer: Authentication is the process of verifying who you are, for example by checking a username and password or a JWT signature. Authorization decides what you are allowed to do after you are authenticated, for example whether you can access `/admin/reports`. Authentication answers the question *"Who is this user?"* while authorization answers *"Is this user allowed to perform this action?"* Both are needed; knowing the user's identity is useless unless the system also enforces access rules.

Q12. What is a JWT, and what are its three parts?

Answer: JWT stands for JSON Web Token. It is a compact, self-contained token used to transmit identity and claims between parties. A JWT has three parts separated by dots: the header, which contains the signing algorithm and token type; the payload, which contains claims such as subject, issuer, expiration, and custom roles; and the signature, which proves the token was issued by a trusted party and was not tampered with. Because the token carries its own claims, a stateless server can verify it without storing session data.

Q13. How do you create and verify a JWT token in a Spring Boot application?

Answer: Creation happens after the user proves their identity, typically in a login endpoint. You build a JWT with a library such as JJWT or Nimbus, set claims like subject, issued-at, expiration, and roles, and sign it with a secret key or a private key. Verification happens in a custom filter or authentication provider that reads the `Authorization` header, parses the token, validates the signature with the same secret or public key, and checks that the token has not expired. If valid, the filter populates the `SecurityContext` with an authenticated `Authentication` object.

Q14. What is the difference between symmetric and asymmetric signing for JWT?

Answer: Symmetric signing uses a single shared secret to both sign and verify the token. It is simple but means every service that verifies tokens must possess the secret, increasing exposure if one service is compromised. Asymmetric signing uses a private key to sign and a public key to verify. The authentication service keeps the private key, while other services only need the public key. Asymmetric signing is preferred in microservices because verification services cannot forge tokens even if they are breached.

Q15. Where should you store the JWT signing key, and why does it matter?

Answer: The signing key must never be hard-coded in source control or plain property files. It should be stored in a secret manager such as HashiCorp Vault, AWS Secrets Manager, Azure Key Vault, or Spring Cloud Config with encryption. For asymmetric keys, the private key stays only in the token issuer, and public keys are distributed to verification services. If a secret key leaks, an attacker can forge valid tokens, which is a complete authentication bypass until all keys are rotated.

Q16. How do you implement login and sign-up using JWT in Spring Boot?

Answer: For sign-up, the controller receives user details, validates them, hashes the password with `BCryptPasswordEncoder`, and saves the user. For login, the controller receives credentials and uses `AuthenticationManager` to validate them. If authentication succeeds, a service creates a JWT containing the user ID, roles, and expiration. The token is returned to the client, which sends it in the `Authorization: Bearer <token>` header on subsequent requests. A custom filter intercepts protected requests, validates the token, and sets the authentication in the `SecurityContext`.

Q17. How does a client use a JWT to access protected endpoints?

Answer: After receiving the JWT from the login response, the client stores it securely, for example in memory or a secure cookie, and sends it in the `Authorization` header as `Bearer <token>` with every protected request. The server extracts the header in a filter, strips the `Bearer` prefix, validates the signature and expiration, and uses the claims to build an `Authentication` object. If the token is missing, malformed, expired, or signed with the wrong key, the server returns 401.

Q18. What is the difference between an access token and a refresh token?

Answer: An access token is short-lived, usually minutes to an hour, and grants access to protected resources. A refresh token is long-lived and is used only to obtain a new access token when the old one expires. The access token travels with every API request, so keeping it short limits the window of misuse if it is stolen. The refresh token is stored more securely, rotated on use, and can be revoked independently. Storing refresh tokens in plain browser storage is a common mistake that makes theft easy.

Q19. How do you implement a refresh token flow securely?

Answer: When the user logs in, issue both an access token and a refresh token. Store a hash of the refresh token in the database along with the user ID, device identifier, and expiration. When the access token expires, the client sends the refresh token to a dedicated refresh endpoint. The server hashes the incoming token, looks it up, verifies it has not expired or been revoked, and issues a new access token and a new refresh token, invalidating the old refresh token. This rotation limits the impact of token theft. Revocation happens by deleting the stored token hash.

Q20. What is session management in Spring Security, and how do you make it stateless?

Answer: Session management controls how user sessions are created and tracked. By default, Spring Security creates a session for browser applications and stores the authentication server-side. For JWT-based REST APIs, you set `sessionCreationPolicy(SessionCreationPolicy.STATELESS)` in the `SecurityFilterChain`. This tells Spring Security not to create or use sessions; every request is authenticated independently using the JWT. Stateless authentication is easier to scale horizontally because no session affinity or shared session store is needed.

Q21. What is the difference between roles and authorities in Spring Security?

Answer: A role is a coarse-grained grouping of permissions, such as `ROLE_ADMIN` or

`ROLE_USER`. An authority is a finer-grained permission, such as `user:read`, `user:write`, or `report:delete`. Internally Spring Security treats both as strings in the `GrantedAuthority` collection. The `hasRole` expression automatically prepends `ROLE_`, so `hasRole("ADMIN")` checks for `ROLE_ADMIN`. `hasAuthority("ADMIN")` checks for the exact string. Using authorities gives more granular control and is preferred when permissions vary across users with the same role.

Q22. How do you implement role-based authorization in Spring Security?

Answer: You can enforce roles at the URL level in the `SecurityFilterChain` using `requestMatchers("/admin/**").hasRole("ADMIN")`. You can also enforce them at the method level with `@PreAuthorize("hasRole('ADMIN')")` or `@Secured("ROLE_ADMIN")`. The user must have the matching role stored in their `GrantedAuthority` list, typically populated from the database when the `UserDetails` object is built. Be careful that `hasRole` adds the `ROLE_` prefix automatically, so storing `ADMIN` in the database and checking `hasRole("ADMIN")` will not match unless you configure the prefix or use `hasAuthority`.

Q23. How do you implement granular authorization with authorities and permissions?

Answer: Granular authorization maps specific actions to authority strings rather than broad roles. For example, an invoice module might use authorities like `invoice:read`, `invoice:write`, and `invoice:approve`. You load these permissions from a database when building the `UserDetails`, and enforce them with `@PreAuthorize("hasAuthority('invoice:approve')")`. For dynamic rules, you can write SpEL expressions that check method arguments, such as `@PreAuthorize("hasAuthority('invoice:write') and #userId == authentication.principal.id")`. This gives much finer control than role-based checks alone.

Q24. What is the difference between `@Secured`, `@PreAuthorize`, and `@PostAuthorize`?

Answer: `@Secured` checks roles before method execution and only supports role names. `@PreAuthorize` is more expressive because it supports SpEL expressions, allowing you to check roles, authorities, method arguments, and custom conditions. `@PostAuthorize` runs after the method executes and can filter the returned object, for example `@PostAuthorize("returnObject.owner == authentication.name")`. `@PreAuthorize` is the most commonly used for method-level authorization. Modern Spring Security uses `@EnableMethodSecurity` instead of the older

@EnableGlobalMethodSecurity.

Q25. Why might method security annotations not work even though they look correct?

Answer: Method security annotations only work when the method is invoked through a Spring-generated proxy. If a method annotated with `@PreAuthorize` is called from another method in the same bean, the proxy is bypassed and the security check is skipped. This is a common and dangerous mistake. The fix is to move the secured method to another bean and inject it, or use `AopContext.currentProxy()`. Also verify that `@EnableMethodSecurity` is present and that the expression syntax matches the actual authority strings.

Q26. What is OAuth2, and how does the Authorization Code flow work?

Answer: OAuth2 is a delegation protocol that lets a user grant a third-party application limited access to their resources without sharing their password. In the Authorization Code flow, the user clicks login, the application redirects the browser to the OAuth2 provider, the user authenticates and consents, and the provider redirects back with an authorization code. The backend exchanges this code for an access token and optionally a refresh token. The access token is then used to call the provider's APIs or to establish an identity. The flow is secure because the user's password never reaches the client application.

Q27. How do you implement Google OAuth2 client authentication in Spring Security?

Answer: Add `spring-boot-starter-oauth2-client` and register Google as a provider in `application.yml` with client ID, client secret, scope, and redirect URI. Expose a `SecurityFilterChain` that uses `oauth2Login()` to handle the Authorization Code flow. Spring Security manages the redirect to Google, the code exchange, and the userinfo call. You can map the returned user details to local application roles using an `OAuth2UserService`. For mobile or single-page apps, use PKCE and keep the client secret on the backend only.

Q28. What is the difference between OAuth2 Login and OAuth2 Resource Server?

Answer: OAuth2 Login makes your application an OAuth2 client that lets users sign in through an external provider such as Google or GitHub. It handles redirects and cookies or sessions. OAuth2 Resource Server makes your application an API that accepts bearer tokens, typically JWT, and validates them. It is used when a mobile app, SPA, or another service calls your API with an access token. A single application can be both a client for login and a resource server for its own APIs.

Q29. How do you handle Spring Security exceptions properly?

Answer: Spring Security throws `AuthenticationException` when the user cannot be authenticated and `AccessDeniedException` when the user lacks permission. You handle these by configuring an `AuthenticationEntryPoint` for 401 responses and an `AccessDeniedHandler` for 403 responses inside the `SecurityFilterChain`. For REST APIs, return structured JSON with a clear message and correlation ID rather than redirecting to an HTML page. A global `@RestControllerAdvice` is useful for application-level exceptions, but security exceptions are caught by the filter chain before they reach the controller.

Q30. How do you debug a 403 Forbidden response in Spring Security?

Answer: Start by checking the logs with `logging.level.org.springframework.security=DEBUG` to see which filter rejected the request and why. Verify the request reached the right `SecurityFilterChain`. Confirm the user is authenticated and that the `Authentication` object contains the expected authorities. Check whether the path matcher is correct and whether `hasRole` or `hasAuthority` matches the stored authority string exactly. If the issue is method-level, ensure the method is being called through a Spring proxy and that `@EnableMethodSecurity` is active.

Q31. What is UserDetailsService, and how is it used?

Answer: `UserDetailsService` is an interface with a single method, `loadUserByUsername`, that Spring Security calls to fetch user data during authentication. Your implementation typically queries the database and returns a `UserDetails` object containing username, password, enabled status, and authorities. This interface separates user retrieval from authentication logic. A common mistake is returning `null` or throwing a generic exception when the user is not found; the correct behavior is to throw `UsernameNotFoundException` so Spring Security can handle it consistently.

Q32. What is PasswordEncoder, and why should you use BCryptPasswordEncoder?

Answer: `PasswordEncoder` is the Spring Security interface for hashing and verifying passwords. `BCryptPasswordEncoder` uses the bcrypt algorithm, which automatically salts and stretches passwords to resist brute-force and rainbow-table attacks. You never store plaintext passwords. During login, the encoder hashes the submitted password and compares it to the stored hash. Bcrypt also has a configurable strength parameter that controls how slow the hash is, letting you increase cost as hardware improves.

Q33. How do you secure passwords and secrets in a Spring Boot application?

Answer: Passwords and secrets must be externalized from source code and configuration files. Use environment variables for non-sensitive values and a secret manager such as Vault, AWS Secrets Manager, Azure Key Vault, or Spring Cloud Config with encryption for credentials. Rotate keys and passwords periodically. For JWT signing keys, use asymmetric keys and keep the private key only in the issuer. Never log secrets, and avoid returning them in Actuator endpoints or error responses.

Q34. What is Remember -Me authentication, and when should you avoid it?

Answer: Remember-Me authentication allows a user to remain logged in across browser sessions using a persistent token stored in a cookie or database. It improves convenience but increases risk because a stolen remember-me cookie can grant long-term access. It should be avoided for sensitive applications such as banking or admin portals. If used, store the token securely, bind it to the user's IP or device fingerprint, and give users a way to revoke all sessions.

Q35. How do you test Spring Security in a Spring Boot application?

Answer: Use `MockMvc` with `@AutoConfigureMockMvc` and `@WithMockUser` to simulate authenticated users. Use `@WithAnonymousUser` for unauthenticated scenarios. Test both the happy path and the denied path by checking the status code. For method security, verify that methods protected by `@PreAuthorize` reject users without the required authority. For JWT, test the filter behavior by generating a valid token in the test and sending it in the `Authorization` header. Integration tests should also verify that the security configuration does not accidentally expose sensitive endpoints.

Q36. What is `HttpSecurity` vs `WebSecurity` in Spring Security?

Answer: `HttpSecurity` configures HTTP-level security such as URL access rules, form login, logout, CSRF, session management, and filters. `WebSecurity` is used for web-level configuration that is not request-specific, such as ignoring certain static resources like CSS, JavaScript, and images so they do not pass through the security filter chain. In Spring Boot 3, `WebSecurityCustomizer` is exposed as a bean for this purpose. Confusing the two can lead to static resources being unnecessarily secured or protected endpoints being accidentally ignored.

Q37. What is the purpose of `ExceptionHandlerFilter`?

Answer: `ExceptionHandlerFilter` sits near the end of the security filter chain and catches `AuthenticationException` and `AccessDeniedException` thrown by other filters or access decision managers. It delegates to the configured

`AuthenticationEntryPoint` to start authentication or to the `AccessDeniedHandler` to report authorization failure. Without this filter, security exceptions would bubble up as raw errors. It is why a missing token results in a 401 and an insufficient role results in a 403.

Q38. What is CORS, and how does it interact with Spring Security?

Answer: CORS, or Cross-Origin Resource Sharing, is a browser mechanism that controls whether a web page can request resources from a different domain. Spring Security can enforce CORS through the `SecurityFilterChain` using `cors()` with a `CorsConfigurationSource` bean. If you enable CORS at the security layer and also at the controller layer, the more restrictive of the two may apply. Allowing all origins with credentials enabled is dangerous because it lets any website make authenticated requests on behalf of users.

Q39. How do you implement logout in a JWT-based stateless application?

Answer: True logout is tricky in a stateless system because the server does not track tokens. The common approach is token blacklisting: store the token or its identifier in a deny list such as Redis until it expires, and check every request against this list. Alternatively, keep access tokens very short-lived and revoke refresh tokens on logout. The client must also delete the token from its storage. Without server-side revocation, a stolen token remains valid until it expires.

Q40. How do you secure service-to-service communication in a microservices architecture?

Answer: Service-to-service calls should use short-lived tokens, mutual TLS, or OAuth2 client credentials. Each service authenticates to a token issuer, receives an access token, and presents it when calling downstream services. The receiving service validates the token and enforces scopes or authorities. Mutual TLS adds an extra layer by verifying both client and server certificates. Use service mesh sidecars such as Istio or Linkerd if you want encryption and identity managed at the infrastructure layer rather than in application code.

Q41. What is an API Gateway's role in security?

Answer: An API Gateway sits in front of microservices and centralizes cross-cutting security concerns such as SSL termination, token validation, rate limiting, IP allow-listing, and request routing. It can validate JWTs or OAuth2 tokens and forward enriched headers to downstream services. However, the gateway is not a substitute for service-level security; each service should still enforce its own authorization rules so that internal traffic or a compromised gateway cannot bypass protection.

Q42. What are the risks of storing JWTs in browser local storage?

Answer: Local storage is accessible to JavaScript, so any XSS vulnerability allows an attacker to steal the token and impersonate the user. The safer approach is to store the access token in memory and keep the refresh token in an `HttpOnly, Secure, SameSite=Strict` cookie. If cookies are not an option, use short-lived access tokens and strict Content Security Policies to reduce the XSS attack surface. Tokens stored in local storage should never be used for highly sensitive applications.

Q43. How do you protect against brute-force login attacks?

Answer: Implement account lockout or progressive delays after repeated failed attempts. Use CAPTCHA after a threshold of failures. Rate-limit the login endpoint by IP or username using a tool such as Bucket4j, Resilience4j, or an API gateway. Log and alert on unusual login patterns. Strong password requirements and `BCryptPasswordEncoder` slow down offline cracking if the database is ever breached.

Q44. What is the principle of least privilege, and how do you apply it in Spring Security?

Answer: The principle of least privilege means giving users and services only the minimum permissions they need. In Spring Security, this translates to using specific authorities instead of broad roles, denying access by default, and granting access only to required endpoints. Instead of giving every user `ROLE_ADMIN`, create fine-grained authorities like `order:read`, `order:write`, and `order:delete`. Regularly audit roles and permissions, and remove unused access during access reviews.

Q45. How do you configure HTTPS for a Spring Boot application?

Answer: Place an SSL certificate and private key in the application or load them from a secret manager. Configure `server.ssl.enabled=true`, `server.ssl.key-store`, `server.ssl.key-store-password`, and related properties. In production, HTTPS is often terminated at a load balancer, reverse proxy, or API gateway rather than the application itself. If termination happens upstream, configure the proxy to forward the original protocol so Spring Security redirects and cookies work correctly.

Q46. What is the difference between `permitAll`, `authenticated`, `anonymous`, and `denyAll`?

Answer: `permitAll` allows any request, including unauthenticated ones. `authenticated` requires a valid authentication. `anonymous` matches requests from users who have not logged in but have been assigned the default anonymous authentication object. `denyAll` blocks every request. These are used to build the access matrix in the `SecurityFilterChain`. A common

error is using `permitAll` on health endpoints but accidentally exposing sensitive data through them.

Q47. How do you handle token expiration gracefully in a client-server JWT flow?

Answer: The server returns 401 with an error indicating the token is expired. The client checks for this specific error and uses the refresh token to request a new access token from a dedicated refresh endpoint. If the refresh token is also expired or revoked, the client redirects the user to login. The server should not extend access token lifetimes automatically, because that defeats the purpose of short-lived tokens and increases the window of misuse if a token is stolen.

Q48. What is `GrantedAuthority`, and how is it different from a role?

Answer: `GrantedAuthority` is the interface that represents a permission assigned to a user, usually implemented as `SimpleGrantedAuthority`. A role is technically an authority that starts with `ROLE_`, and `hasRole` expects that prefix. An authority is any permission string. The difference is mostly convention: roles are broad, authorities are fine-grained. You can store both in the same collection and check them with `hasRole`, `hasAuthority`, or custom SpEL.

Q49. How do you implement multi-factor authentication in Spring Security?

Answer: Multi-factor authentication requires a second factor beyond the password. After the user enters the password, the system verifies the second factor, such as a TOTP code from an authenticator app, an SMS code, or a hardware key. In Spring Security, this is usually implemented with a custom authentication flow that stores a partially authenticated principal in the session, prompts for the second factor, and then marks the authentication as fully authenticated. Commercial identity providers such as Keycloak or Auth0 handle this out of the box and can be integrated as an OAuth2 provider.

Q50. What are the most important security checks before releasing a Spring Boot application?

Answer: Before release, ensure HTTPS is enforced, CSRF is configured correctly for the client type, security headers are set, secrets are externalized, passwords are hashed with bcrypt, Actuator endpoints are restricted, error responses do not leak stack traces, JWT tokens are short-lived with secure key storage, method-level security is tested, and dependencies are scanned for known vulnerabilities. Perform penetration testing and review the security configuration in code review. Security is a continuous process, not a one-time checklist.

Scenario based questions

S1. A user reports they can access the admin dashboard even though they are not an admin. How do you investigate?

Answer: First I verify the endpoint is actually protected in the `SecurityFilterChain` and that the user does not have the admin role due to a data issue. I check the authentication object in the logs or a debug endpoint to see the granted authorities. I look for bugs in the role assignment logic, such as assigning `ROLE_ADMIN` to all users during sign-up or failing to strip legacy roles after a promotion. I also check whether method-level security is being bypassed by self-invocation from within the same bean. The fix is to tighten the URL and method authorization rules, add tests for non-admin access, and audit the user-role mapping code.

S2. A penetration test report says your login endpoint is vulnerable to SQL injection. How do you respond?

Answer: I immediately confirm whether the endpoint uses parameterized queries or an ORM. If the login query concatenates the username or password into SQL, I rewrite it to use named parameters or the JPA/Criteria API. I add server-side input validation and review all other endpoints for similar patterns. I run static analysis tools and add tests that pass malicious payloads such as `' OR '1'='1`. Finally, I verify database accounts use least privilege so that even if injection occurs, the damage is limited.

S3. A developer disables CSRF to make a React frontend work, and now the security team is concerned. What do you say?

Answer: Disabling CSRF globally is only safe for stateless APIs that do not rely on session cookies. If the React app uses session cookies for authentication, disabling CSRF exposes the application to cross-site request forgery. The proper fix is to keep CSRF enabled and provide the token to the frontend, either as a cookie readable by JavaScript or through an endpoint, and have the React app send it back in a header for state-changing requests. Alternatively, switch to JWT in an `HttpOnly` cookie or an `Authorization` header, where CSRF is not relevant.

S4. A JWT token works in dev but is rejected in production with a 401. How do you debug?

Answer: I check whether the production signing key matches the one used to sign the token. If the token was issued in dev and production uses a different secret, it will fail validation. I verify the token has not expired and that the issuer and audience claims match the production configuration. I also check the clock skew between the issuer and verifier, and I confirm the filter reads the `Authorization` header correctly without being blocked by a proxy or gateway. The fix is to align key management and claims validation across environments, ideally using a single issuer or public key distribution.

S5. A user logged out, but their JWT is still able to access protected resources. How do you fix this?

Answer: In a stateless JWT system, logout only works if the server keeps a deny list of tokens until they expire. I implement token blacklisting using Redis or a database, storing the token identifier or a hash keyed by expiration time. A custom filter checks every request against this list and rejects blacklisted tokens. Alternatively, I keep access tokens very short-lived and revoke refresh tokens on logout, so the user must re-authenticate when the short token expires. The client must also delete tokens from storage.

S6. A production API returns a 500 error with a full stack trace containing the database password. How do you prevent this?

Answer: A stack trace with secrets means an unhandled exception leaked through the default error response. I add a global `@RestControllerAdvice` that catches exceptions and returns a sanitized JSON response with only a generic message and a correlation ID. I set `server.error.include-stacktrace=never` and `server.error.include-message=never` in production. I ensure no secrets are placed in exception messages. Finally, I review the error logging pipeline to make sure full stack traces go to the centralized logging system, not the API response.

S7. A Spring Security filter you added is being ignored. What could be wrong?

Answer: Custom filters must be registered at the correct position in the filter chain using `addFilterBefore`, `addFilterAfter`, or `addFilterAt`. If the filter is added at the wrong position, authentication or authorization may run before it, causing the request to be processed or rejected before your filter executes. Also, the filter must be a Spring bean if it depends on other beans, but it should not be registered as a servlet filter automatically unless you want it to run twice. I verify the filter order in the logs with debug logging enabled and adjust the registration accordingly.

S8. A third-party OAuth2 provider integration is failing after a certificate rotation. How do you investigate?

Answer: I check the TLS truststore used by the JVM to verify the provider's new certificate. If the certificate chain is not trusted, the token exchange or userinfo call fails with a certificate validation error. I update the truststore with the new issuer certificates or switch to a well-known CA bundle. I also verify the client ID, client secret, redirect URI, and scopes match the provider's current configuration. Finally, I confirm that the token issuer and userinfo endpoints have not changed.

S9. A `@PreAuthorize` annotation on a service method does not work when called from the controller. Why?

Answer: If the method is called from another method in the same bean, the Spring proxy is bypassed and the annotation is ignored. This is a classic self-invocation problem. I move the annotated method to another bean and inject it, or use `AopContext.currentProxy()` to force a proxied call. I also verify that `@EnableMethodSecurity` is present and that the SpEL expression uses the correct role or authority names, remembering that `hasRole` adds the `ROLE_` prefix.

S10. Your application uses Google OAuth2 login, but users are getting assigned the same default role. How do you fix role mapping?

Answer: By default, Spring Security assigns limited authorities based on the OAuth2 provider response. I implement a custom `OAuth2UserService` that calls Google's `userinfo` endpoint and maps returned attributes to local roles stored in the database. For example, I might check a domain, email pattern, or a pre-registered admin list. I then build a new `OAuth2User` with the mapped `GrantedAuthority` collection. I also ensure that roles are not hard-coded and can be updated without redeploying the application.

S11. A load test shows that JWT validation is becoming a CPU bottleneck. How do you optimize it?

Answer: I first check whether validation uses symmetric key signing with a shared secret that requires expensive HMAC operations on every request, and whether the token is parsed from scratch each time. Switching to an asymmetric key pair lets verification services use only the public key, which is faster and safer. I cache the parsed token or the decoded public key. I also verify the token format before parsing to fail fast on malformed tokens. If the bottleneck persists, I move token validation to an API Gateway or sidecar so backend services receive pre-validated claims.

S12. A user changed their password, but old sessions on other devices remain active. How do you handle this?

Answer: For session-based applications, I invalidate all sessions associated with the user by using Spring Security's session registry, specifically for concurrent session control. For JWT-based applications, I issue new access and refresh tokens and revoke or blacklist all existing refresh tokens for that user. The client must re-authenticate on other devices. I also add a `tokenVersion` or `passwordChangedAt` claim in the JWT and reject tokens issued before the password change.

S13. A penetration test found that the refresh token endpoint does not rotate tokens. How do you fix it?

Answer: Token rotation means issuing a new refresh token every time a refresh request is made and

invalidating the old one. I update the refresh endpoint to verify the incoming refresh token, delete or mark the stored token as used, and issue a new access token along with a new refresh token. This limits the window in which a stolen refresh token can be used. I also bind refresh tokens to the device fingerprint or a rotating family identifier so that reuse of an old token invalidates the entire family, preventing silent token theft.

S14. A mobile app reports that tokens expire too quickly, causing frequent login prompts. What is the balance?

Answer: The balance is between security and user experience. Access tokens should remain short, for example 5 to 15 minutes, because they travel with every request. Refresh tokens can live longer, for example days or weeks, but must be stored securely and rotated. For mobile apps, using a secure enclave or encrypted storage for the refresh token and implementing silent refresh on 401 responses gives a smooth experience without sacrificing safety. Long-lived access tokens are not the right answer because theft becomes much more dangerous.

S15. A developer stores the JWT secret key in `application.properties` and checks it into Git. What do you recommend?

Answer: I treat the exposed secret as compromised and rotate it immediately. The secret key must be removed from source control and moved to a secret manager such as Vault, AWS Secrets Manager, or Azure Key Vault. For microservices, I recommend switching to asymmetric signing so verification services only need the public key. I add the `properties` file to `.gitignore`, scan the repository history for other secrets, and implement pre-commit hooks to prevent future commits of credentials.

S16. A state-changing API does not check user ownership and allows modifying any record by ID. How do you fix this?

Answer: This is an insecure direct object reference and missing authorization. The fix is to add an ownership check inside the service layer, for example by comparing the record's owner to the current principal. Use `@PreAuthorize` with a SpEL expression that checks the owner, or perform the check in code after loading the entity from the database. Never trust the client to send the correct ID without verifying it against the authenticated user. I also add tests that confirm a user cannot update another user's records.

S17. A production incident shows brute-force attempts on the login endpoint. How do you respond?

Answer: I immediately implement rate limiting on the login endpoint by IP address and username using Bucket4j, Resilience4j, or the API Gateway. I add progressive delays and CAPTCHA after repeated failures. I log and alert on attack patterns and consider temporarily blocking IPs at the

firewall or WAF level. I verify that passwords are hashed with `BCryptPasswordEncoder` so offline cracking is slow if the database is breached. I also enable monitoring for credential-stuffing patterns and encourage users with weak or reused passwords to change them.

S18. A frontend developer reports CORS errors after enabling Spring Security. How do you resolve them without weakening security?

Answer: CORS errors happen when the browser blocks a cross-origin request. I configure CORS in the `SecurityFilterChain` using a `CorsConfigurationSource` bean that defines allowed origins, methods, headers, and whether credentials are allowed. I do not use a wildcard origin when credentials are enabled, because that allows any website to make authenticated requests. Instead, I explicitly list trusted origins and keep allowed methods minimal. I also ensure the preflight `OPTIONS` request is permitted without authentication.

S19. A JWT-based API needs to revoke access for a user immediately due to a security incident. How do you do it?

Answer: Because JWTs are stateless, I cannot revoke an access token directly unless it is expired. I implement token blacklisting using Redis keyed by token identifier or expiration time, and I update the authentication filter to reject blacklisted tokens. For faster response, I keep access tokens very short-lived and revoke all refresh tokens for the user. I also update the user record to mark the account as locked or disabled so that any new login attempt fails. If the incident is severe, I rotate the signing key, which invalidates all existing tokens across the system.

S20. An internal service calls your API with a service account token. How do you authorize it differently from a human user?

Answer: I distinguish service accounts from human users by claim values such as `client_id`, `sub`, or a custom `account_type` claim. Service accounts get authorities that represent API scopes or permissions, not UI roles. I use `@PreAuthorize("hasAuthority('service:invoice:read')")` to restrict service endpoints and keep service account tokens short-lived, ideally with a lifetime tied to the request. I also log service-to-service calls separately and monitor for unusual call volumes or scope creep.

S21. A developer uses `permitAll` on the `/actuator/health` endpoint but it still returns 401. Why?

Answer: The endpoint might be configured in the wrong `SecurityFilterChain` or the request might be hitting a different base path. If management endpoints are on a separate port or context path, the main `SecurityFilterChain` may not apply. Another cause is that the request is being rejected by an earlier filter, such as a custom JWT filter that returns 401 for any request without a token before the authorization filter can evaluate `permitAll`. I debug by enabling

Spring Security logging and checking the exact chain that processes the request.

S22. A user role hierarchy should allow managers to do everything users can. How do you model this?

Answer: Spring Security supports role hierarchies through a `RoleHierarchy` bean. I define that `ROLE_MANAGER` implies `ROLE_USER`, and then use `hasRole("USER")` in authorization rules. A manager will automatically be granted the implied authority. This avoids duplicating permission checks across roles and keeps the authority model maintainable. Without role hierarchy, I would have to explicitly check both roles in every rule, which is error-prone.

S23. A Swagger UI is publicly exposed in production and the security team wants it removed. What is the best approach?

Answer: The best approach is to disable Swagger UI in production using Spring profiles, for example by only enabling the `SpringDoc` configuration when the `dev` or `qa` profile is active. Alternatively, I restrict Swagger endpoints with Spring Security to internal users only. If Swagger is not needed in production at all, I remove the dependency from the production build. I keep OpenAPI spec generation in CI so documentation stays accurate, but the interactive UI is not reachable publicly.

S24. A user with a valid token sees 403 on some endpoints after a deployment. What do you check?

Answer: I check whether the deployment changed authority strings or role mappings. For example, a role renamed from `ADMIN` to `ROLE_ADMIN` or an authority renamed from `user.write` to `user:create` breaks existing `@PreAuthorize` expressions. I verify the `UserDetailsService` or token claims builder populates the same authorities the rules expect. I also check whether the endpoints moved to a new path that is no longer covered by the URL authorization rule. Logging the authentication object and the denied request path quickly shows the mismatch.

S25. A Spring Security application must support both human users and API clients. How do you configure it?

Answer: I configure multiple `SecurityFilterChain` beans with different request matchers and authentication mechanisms. One chain handles browser users with OAuth2 login or form login and session cookies. Another chain handles API clients with JWT bearer tokens and a stateless session policy. Each chain matches based on path or header patterns. This separation prevents a misconfiguration meant for browser users, such as CSRF protection, from breaking API clients, and vice versa. Both chains still enforce authorization based on roles and authorities.